

Lesson 12: Streams & Real-time Data

One value is not always enough

A Future gives you one value, one time. But some data keeps coming: chat messages, live cricket scores, a stopwatch ticking every second.

For data that arrives again and again, Dart has a Stream. Think of it this way:

- Future = one parcel delivered to your door.
- Stream = a pipe with water flowing through it, nonstop.

Making a stream

Here is a stream that gives a new number every second:

```
Stream<int> countSeconds() async* {  
  int i = 0;  
  while (true) {  
    await Future.delayed(const Duration(seconds: 1));  
    yield i++; // "yield" sends a value down the stream  
  }  
}
```

Two new keywords:

- `async*` (async-star) -- marks a function as "this gives many values over time."
- `yield` -- sends one value down the stream, then keeps going.

Call it once, and numbers 0, 1, 2, 3... keep arriving every second until you stop listening.

Listening with StreamBuilder

Flutter's StreamBuilder widget rebuilds itself every time a new value arrives:

```
StreamBuilder<int>(  
  stream: countSeconds(),  
  builder: (context, snapshot) {  
    if (!snapshot.hasData) {  
      return const Text('Starting...');  
    }  
    return Text('Seconds: ${snapshot.data}');  
  },  
)
```

snapshots is just a wrapper around the latest value. `snapshot.hasData` is false before the first value arrives, and `snapshot.data` holds the newest number. The text on screen updates every second -- with no `setState()` needed.

Compare with Lesson 10: `FutureBuilder` does the same job for a single `Future`. Same idea, different shape of data.

Stopping the flow

A stream that never ends will run forever -- even after the user leaves the screen. That wastes battery and memory.

`StreamBuilder` handles this for you: when the widget is removed from the screen, it stops listening on its own. But if you listen to a stream manually with `.listen()`, you must cancel it yourself:

```
late StreamSubscription sub;

@override
void initState() {
  super.initState();
  sub = countSeconds().listen((n) {
    print('Tick: $n');
  });
}

@override
void dispose() {
  sub.cancel(); // always cancel manual listeners
  super.dispose();
}
```

Rule of thumb: prefer `StreamBuilder` -- it cleans up after itself.

Where streams are used

You will meet streams in many real apps:

- Chat apps -- new messages arrive as a stream.
- Live scores -- the score updates without refreshing.
- Firebase -- Firestore sends live updates through streams (you will use this in Lesson 16).
- Search-as-you-type -- each keystroke is a new value in a stream.

Whenever data changes over time on its own, a stream is probably the right tool.

Key terms

- Stream -- a flow of values arriving over time (unlike a Future's single value).
- async* -- marks a function that produces a stream.
- yield -- sends one value down a stream.
- StreamBuilder -- a widget that rebuilds every time a new stream value arrives.
- snapshot -- wrapper holding the latest stream value (snapshot.data).
- cancel() -- stops listening to a stream; always cancel manual listeners.